Divide and Conquer, Part 3

Karatsuba's Algorithm, Linear Time Selection

Karatsuba's Algorithm

Multiplying Big Integers

Multiplying two 32- or 64-bit integers is a single hardware instruction.

But for integers with millions of digits, the grade-school algorithm's $O(n^2)$ starts to hurt.

$$\begin{array}{r} 1\ 0\ 1\ 1 \\ \times\ 1\ 1\ 0\ 1 \\ \hline \end{array}$$

n rows of n digits

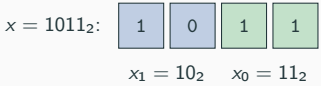
$O(n^2)$ digit operations

Can we do better?

Splitting the Inputs

Write each n-bit number as a high half and a low half, with $m = \lfloor n/2 \rfloor$:

$$x = x_1 \cdot 2^m + x_0, \qquad y = y_1 \cdot 2^m + y_0.$$



Expanding directly:

$$x \cdot y = x_1 y_1 \cdot 2^{2m} + (x_1 y_0 + x_0 y_1) \cdot 2^m + x_0 y_0.$$

The Naive Split Gains Nothing

That expansion needs four half-size multiplications:

$$x_1 y_1, \quad x_1 y_0, \quad x_0 y_1, \quad x_0 y_0.$$

Additions and shifts are $O(n)$, so

$$T(n) = 4\,T(n/2) + O(n) \overset{\text{Master}}{=} O(n^2).$$

Exactly what we started with.

Karatsuba's Trick

The middle coefficient needs only one extra multiplication. Define

$$P_1 = x_1 y_1, \qquad P_2 = x_0 y_0, \qquad P_3 = (x_1 + x_0)(y_1 + y_0).$$

Then

$$x_1 y_0 + x_0 y_1 = P_3 - P_1 - P_2,$$

and therefore

$$x \cdot y = P_1 \cdot 2^{2m} + (P_3 - P_1 - P_2) \cdot 2^m + P_2.$$

Three multiplications instead of four.

Exercise

Verify that $P_3 - P_1 - P_2 = x_1 y_0 + x_0 y_1$ by expanding P_3.

Solution

Expand the product:

$$P_3 = (x_1 + x_0)(y_1 + y_0) = \underbrace{x_1 y_1}_{P_1} + x_1 y_0 + x_0 y_1 + \underbrace{x_0 y_0}_{P_2}.$$

Subtracting P_1 and P_2 leaves exactly the middle terms:

$$P_3 - P_1 - P_2 = x_1 y_0 + x_0 y_1. \qquad \square$$

We trade one multiplication for a few extra additions, which is a good deal, since additions are only $O(n)$.

Running Time

Three half-size multiplications plus $O(n)$ of additions and shifts:

$$T(n) = 3\,T(n/2) + O(n)$$

Master theorem with $a = 3$, $b = 2$, $d = 1$. Since $\log_2 3 \approx 1.585 > 1$:

$$T(n) = O(n^{\log_2 3}) \approx O(n^{1.585}).$$

A real improvement over $O(n^2)$ for large n.

Implementation

```
function karatsuba(x::BigInt, y::BigInt)
    # Base case: built-in multiplication for small numbers
    if x < 4 || y < 4
        return x * y
    end

    n = max(ndigits(x, base=2), ndigits(y, base=2))
    m = n div 2

    # Split x = x1 * 2^m + x0, y = y1 * 2^m + y0
    x1, x0 = x >> m, x & ((BigInt(1) << m) - 1)
    y1, y0 = y >> m, y & ((BigInt(1) << m) - 1)

    # Three recursive multiplications
    P1 = karatsuba(x1, y1)
    P2 = karatsuba(x0, y0)
    P3 = karatsuba(x1 + x0, y1 + y0)

    return (P1 << 2m) + ((P3 - P1 - P2) << m) + P2
end
```

Linear Time Selection

The Selection Problem

Given an unsorted array $A[1:n]$ and k, find the k-th smallest element.

- $k = 1$: the minimum $\quad k = n$: the maximum
- $k = \lceil n/2 \rceil$: the median

Sorting gives $O(n \log n)$. Can we do better?

Quickselect

Pick a pivot p and partition:

- $k \leq |L|$: recurse into L.
- $k \leq |L| + |E|$: the answer is p.
- otherwise: recurse into G looking for the $(k - |L| - |E|)$-th smallest.

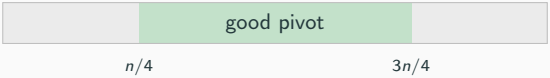
Only one side is searched, unlike quicksort.

Quickselect

```
function quickselect(A, k)
    n = length(A)
    if n == 1
        return A[1]
    end

    p = A[rand(1:n)] # random pivot

    L = [a for a in A if a < p]
    E = [a for a in A if a == p]
    G = [a for a in A if a > p]

    if k <= length(L)
        return quickselect(L, k)
    elseif k <= length(L) + length(E)
        return p
    else
        return quickselect(G, k - length(L) - length(E))
    end
end
```

Quickselect: Expected Time

With probability $1/2$ the random pivot lands between the $n/4$-th and $3n/4$-th smallest, so both L and G have at most $3n/4$ elements.



In expectation the size shrinks by $3/4$ every couple of calls:

$$T(n) = T(3n/4) + O(n) \;=\; O(n).$$

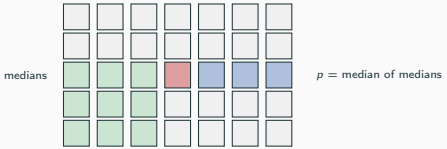
Expected $O(n)$, but the worst case is still $O(n^2)$.

Median of Medians

Choose a pivot that is guaranteed to split well (Blum, Floyd, Pratt, Rivest, Tarjan).

1. Divide A into groups of 5.
2. Find the median of each group by brute force.
3. Recursively find the median p of those $\lceil n/5 \rceil$ medians.
4. Partition on p and recurse as in quickselect.

Why the Pivot Is Good



groups sorted top to bottom, medians in the middle row

Claim

At least $\approx \frac{3n}{10}$ elements are $\leq p$, and at least $\approx \frac{3n}{10}$ are $\geq p$.

Proof Sketch

There are $\lceil n/5 \rceil$ medians, and at least half of them are $\leq p$.

For each such median, at least 3 of the 5 elements in its group are $\leq p$, namely the median itself and the two below it.

That gives at least

$$3\left\lfloor \frac{1}{2}\left\lceil \frac{n}{5} \right\rceil \right\rfloor \approx \frac{3n}{10}$$

elements $\leq p$. Symmetrically for $\geq p$. $\square$

So the recursive call runs on at most $\approx \frac{7n}{10}$ elements.

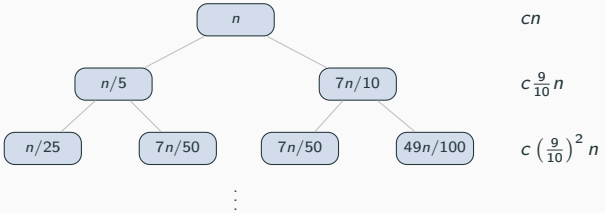
The Recurrence

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

- $T(n/5)$: finding the median of the medians (step 3).
- $T(7n/10)$: the selection call on L or G (step 4).
- $O(n)$: grouping, group medians, and partitioning.

Note $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, which is what makes it linear.

Why It Is $O(n)$



$$cn + c\left(\frac{9}{10}\right)n + c\left(\frac{9}{10}\right)^2 n + \cdots = cn\sum_{i=0}^{\infty}\left(\frac{9}{10}\right)^i = 10cn = O(n).$$

Exercise

What happens with groups of 3 or 7 instead of 5?

Write out the recurrence in each case and decide whether the algorithm is still $O(n)$.

Solution

Groups of 3. At least 2 of every 3 elements in half the groups are $\leq p$, giving $\approx \frac{2n}{6} = \frac{n}{3}$ on each side, so the recursion is on $\leq \frac{2n}{3}$:

$$T(n) = T\left(\tfrac{n}{3}\right) + T\left(\tfrac{2n}{3}\right) + O(n), \qquad \tfrac{1}{3} + \tfrac{2}{3} = 1.$$

The level work no longer shrinks: every level costs cn, and there are $\Theta(\log n)$ of them: $T(n) = \Theta(n \log n)$.

Groups of 7. At least 4 of every 7 in half the groups, giving $\approx \frac{4n}{14} = \frac{2n}{7}$ per side:

$$T(n) = T\left(\tfrac{n}{7}\right) + T\left(\tfrac{5n}{7}\right) + O(n), \qquad \tfrac{1}{7} + \tfrac{5}{7} = \tfrac{6}{7} < 1 \implies O(n).$$

Any odd group size ≥ 5 works; 3 is the one that fails.

Implementation

```
function median_of_medians(A, k)
    n = length(A)

    # Base case: sort and return the k-th smallest
    if n <= 10
        return sort(A)[k]
    end

    # Steps 1-2: groups of 5, median of each
    medians = [sort(A[i:i+4])[3] for i in 1:5:n-n%5-4]

    # Step 3: recursively find the median of medians
    p = median_of_medians(medians, (length(medians) + 1) div 2)

    # Step 4: partition and recurse
    L = [a for a in A if a < p]
    E = [a for a in A if a == p]
    G = [a for a in A if a > p]

    if k <= length(L)
```

In Practice

	worst case	in practice
Sorting	$O(n \log n)$	simple, often fine
Quickselect	$O(n^2)$	expected $O(n)$, small constants
Median of medians	$O(n)$	large constants

The linear-time guarantee is beautiful, but quickselect with a random pivot is usually the faster choice.

Questions?